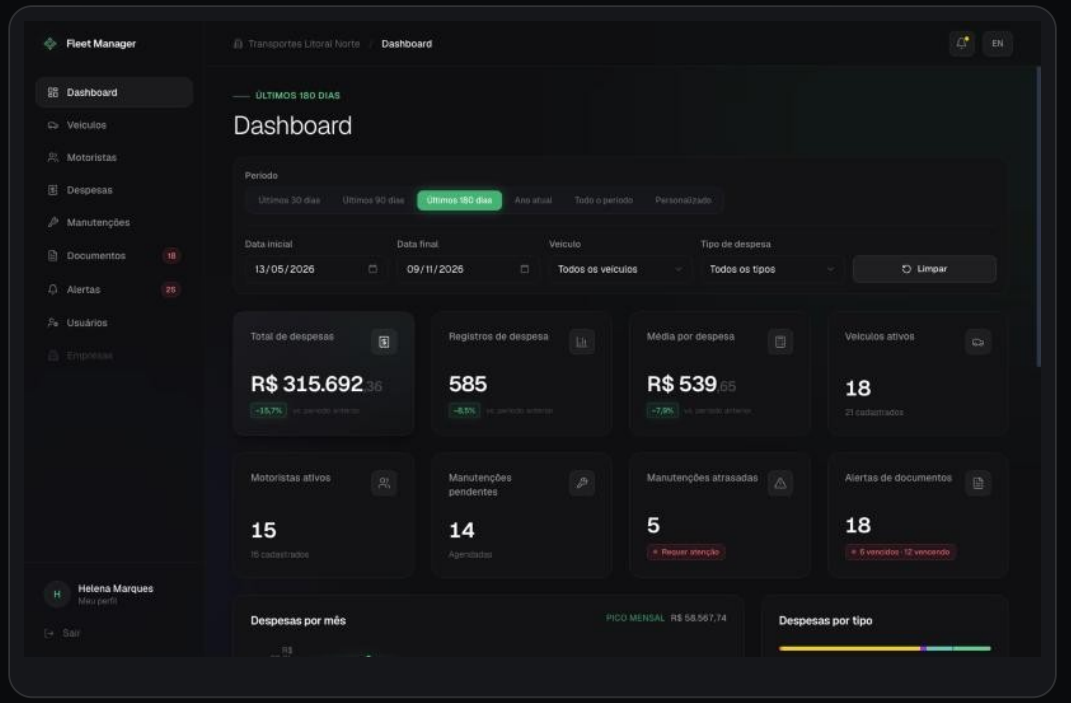
Fleet Manager

Sistema web de gestão de frotas

André Luiz Cavalcante da Silva
Luiz Eduardo Pacheco
Gregory Figueiredo de M. P. Jereissati



pfi-fleet-manager-api.vercel.app

Orientador: Prof. Me. Ronaldo Gonçalves Junior
Ciência da Computação · Universidade de Fortaleza · 2026

O problema e o que entregamos

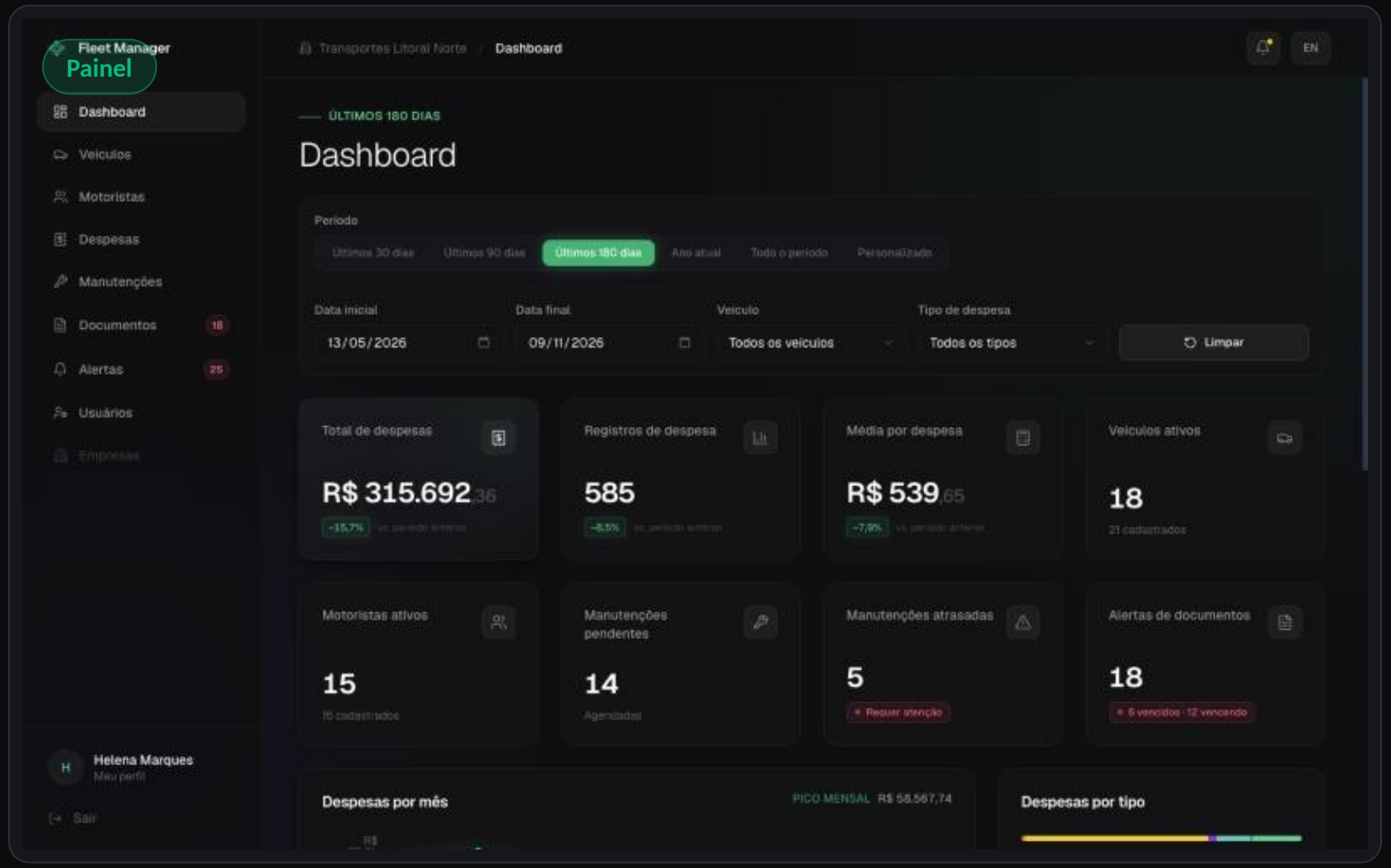
Antes

- Despesas, manutenções e documentos em planilhas e pessoas diferentes
- Ninguém sabe quanto cada veículo custa
- A planilha não avisa: o documento vence e vira multa

Fleet Manager

- Uma base única, em que todo lançamento pertence a um veículo
- Custo por veículo e por período, num painel
- Prazos de documentos, manutenções e CNH calculados e avisados numa central
- No ar, para várias empresas, cada uma enxergando só o que é seu

Demonstração: o painel de indicadores



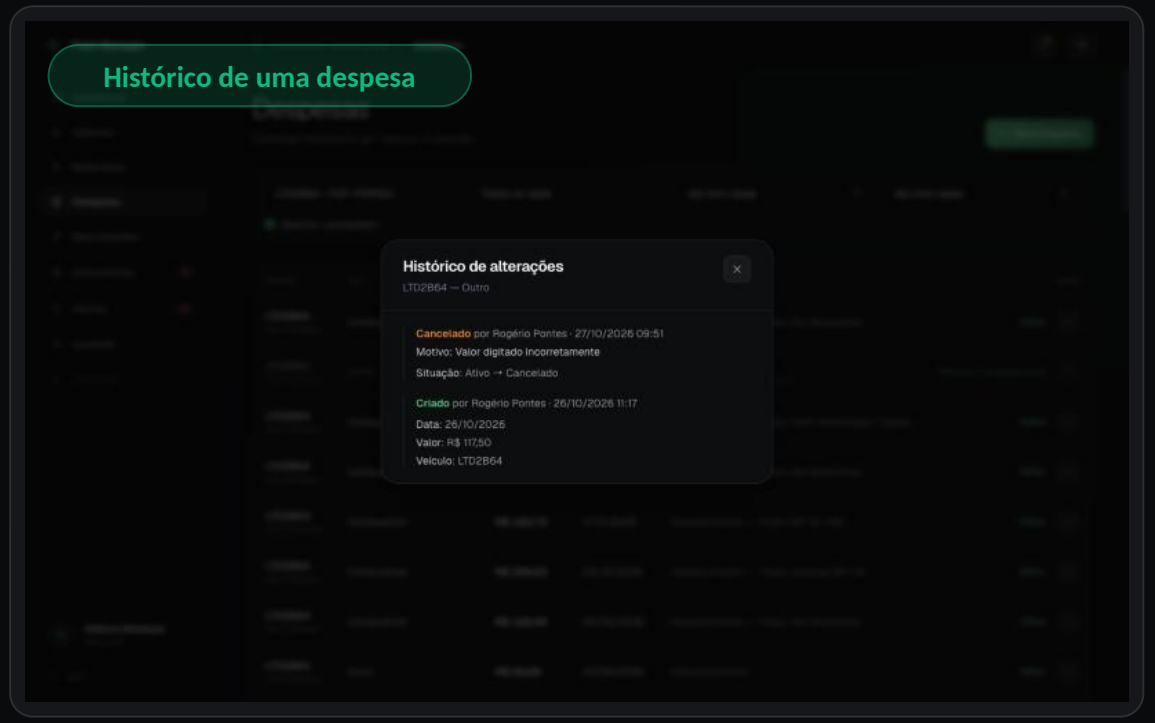
- Custo total, médio e por veículo
- Variação contra o período anterior, de mesma duração
- Filtro por período, veículo e categoria
- Somas feitas no banco: só os totais trafegam

Demonstração: alertas sem sair da tela



- O sino mostra os três alertas mais urgentes
- Documentos e CNH: até 30 dias; manutenções: até 7
- A manutenção atrasada aparece sem ninguém marcar
- A rotina diária registra quando sinalizou cada documento

Demonstração: as regras na prática



- Lançar, recusar data futura, cancelar com motivo e ver o histórico



- CNH vencida recusada no vínculo; motorista vê só o que é dele

Resultados em números

402

testes automatizados

Em três níveis, com 88,4% das linhas do servidor executadas

0 erros

em 124.753 requisições

Workload misto com até 100 conexões; p99 sempre abaixo de 350 ms

3 s

para avisar que o banco caiu

Responde 503 em vez de travar e volta sozinho, sem reiniciar

50 mil

despesas numa empresa

A lista abre em 17 ms e a tela em menos de 1 s

2 min

de testes a cada mudança

Teste vermelho não vai ao ar: a publicação espera o CI

0

vulnerabilidades conhecidas

Auditoria das dependências limpa

Os sete objetivos, cumpridos e comprovados

Objetivo	Evidência
1 Cadastro e vínculos	Testes de veículos e cadastro; seções 4.4 a 4.6
2 Despesas em seis categorias	Testes de despesas e de precisão monetária
3 Manutenções e atraso	Atraso calculado, testado contra o banco
4 Documentos e alertas	Central com três frentes; Figura 9
5 Painel de indicadores	Figura 8; tempos com 50 mil despesas
6 Autenticação e acesso	48 testes de autenticação, permissão e isolamento
7 Qualidade, desempenho e resiliência	402 testes, CI, workload e queda do banco

Limitações, declaradas, e próximos passos

O que não foi feito

- Não foi aplicado numa frota real: os benefícios são expectativas fundamentadas
- Testes de tela feitos, mas fora da integração contínua
- Carga local com gerador, API e banco na mesma máquina
- Validade da CNH em dois lugares, na ficha e no documento

Próximos passos

- Validação com gestores de frota reais, com medição antes e depois
- Testes de tela na integração contínua
- Unificar a validade da CNH
- Notificação por e-mail, hoje fora do escopo

Obrigado

Perguntas?

pfi-fleet-manager-api.vercel.app
github.com/gregoryjereissati/pfi-fleet-manager

Apoio para perguntas

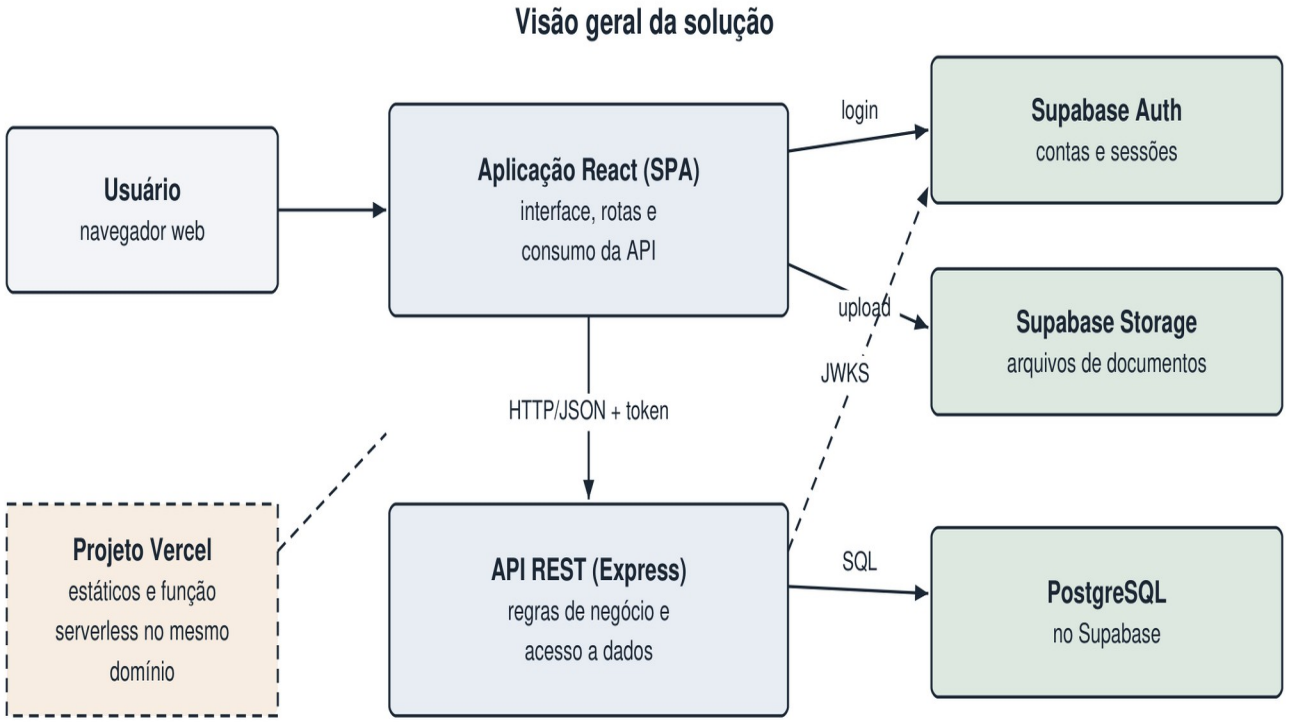
Slides técnicos para abrir se a banca perguntar. Não fazem parte dos 10 minutos.

Objetivo: centralizar a frota e avisar antes do prazo

Objetivo geral sistema web para o controle operacional, financeiro e documental de veículos, com custo por ativo, classificação automática de vencimentos e redução de riscos legais

1. Cadastro de veículos, motoristas e usuários, com vínculo
2. Despesas por veículo, em seis categorias
3. Manutenções, com o atraso apurado sozinho
4. Documentos com vencimento e central de alertas
5. Painel de indicadores de custo
6. Autenticação e controle de acesso por papel
7. Qualidade verificada em três níveis, com integração contínua, desempenho e resiliência

A solução: uma base única, em torno do veículo



Interface
React e TypeScript, no design system Ledger

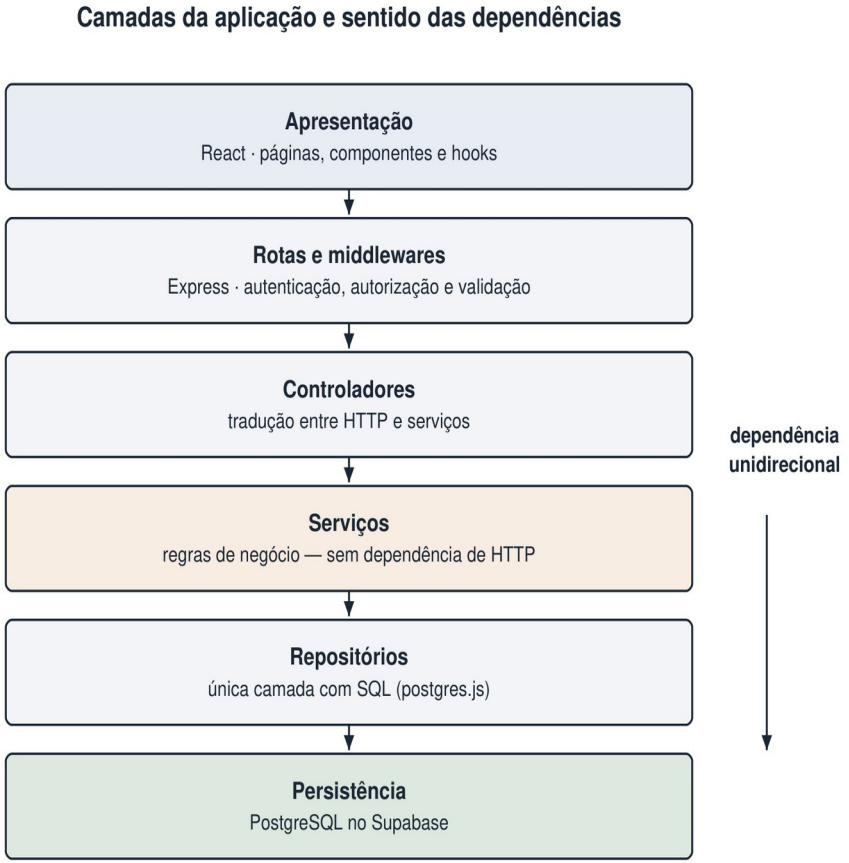
Servidor
Node.js com Express, em camadas

Dados
PostgreSQL no Supabase, sem ORM

Acesso
Supabase Auth e controle por papel

Publicação
Vercel, interface e API no mesmo domínio

Arquitetura em camadas, num só sentido

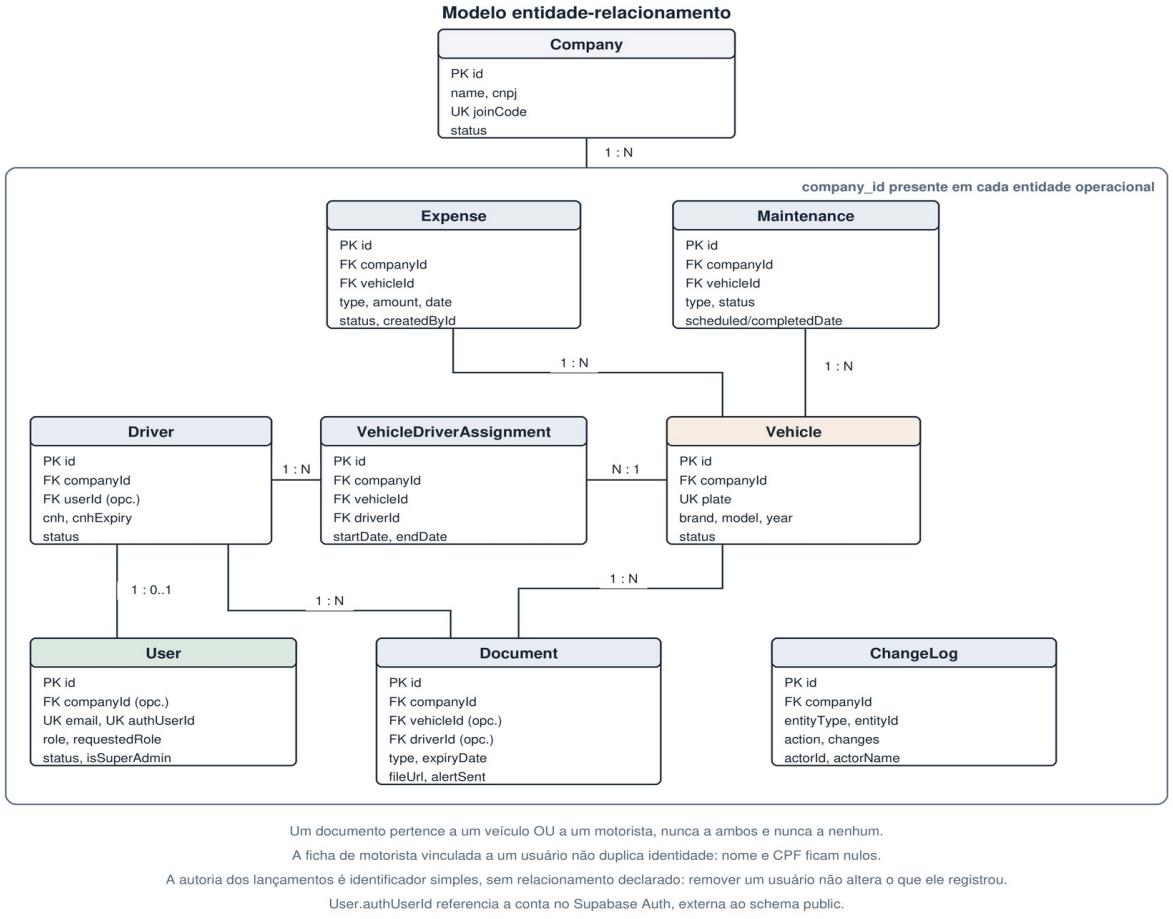


1 **Serviços não conhecem o HTTP**
As regras de negócio são testadas sem subir servidor nem banco

2 **SQL parametrizado, sem ORM**
A condição de empresa fica visível em cada consulta; o custo é escrever mais

3 **Tipos compartilhados**
Mudar um contrato entre interface e servidor vira erro de compilação

Dados: a empresa é a fronteira de tudo



companyId em toda entidade

Toda consulta filtra pela empresa do perfil, lida do banco — nunca do que o navegador envia

Situações calculadas, nunca gravadas

Documento vencido, CNH vencida e manutenção atrasada saem da data a cada consulta

Migrations

Arquivos SQL numerados que mudam o esquema, aplicados em ordem; só se acrescenta, nunca se altera um já aplicado

Segurança: quem vê o quê é decidido no servidor

	Administrador	Gestor	Motorista
Aprovar contas e mudar papéis	Sim	Não	Não
Cadastrar veículos e documentos	Sim	Sim	Não
Lançar despesa ou manutenção	Empresa	Empresa	Só no veículo vinculado
Ver lançamentos e alertas	Empresa	Empresa	Só os próprios
Excluir veículo em definitivo	Sim	Não	Não

Outra empresa: o registro responde como inexistente (404), sem revelar que existe

Token ES256
Verificado com a chave pública do Supabase; a API não guarda senha nem segredo

Bloqueio imediato
Papel e situação ficam no banco e são relidos a cada requisição

Nunca sem administrador
Rebaixar, bloquear ou excluir o último administrador é recusado

Regras de negócio que trabalham pelo gestor

Atraso calculado

A manutenção vira atrasada pela data; ninguém precisa lembrar de marcar

CNH vencida

Não assume veículo novo; o vínculo existente fica com o gestor decidir

Despesa sem data futura

Despesa é gasto realizado; o previsto é manutenção programada

Cancelar com motivo

O lançamento errado sai dos totais, mas continua visível com o porquê

Sempre um administrador

A empresa não fica impedida de aprovar contas

Motorista vê o que é seu

Privacidade entre colegas e menos chance de erro

São 23 regras, cada uma com o motivo de existir — Quadro 7 do documento

Verificação em oito camadas

1. Análise estática
2. Unitários
3. Integração com o banco
4. Ponta a ponta
5. Integração contínua
6. Carga e volume
7. Resiliência
8. Produção

402

testes automatizados

213 unitários · 52 de integração · 137 de ponta a ponta

88,4%

das linhas do servidor

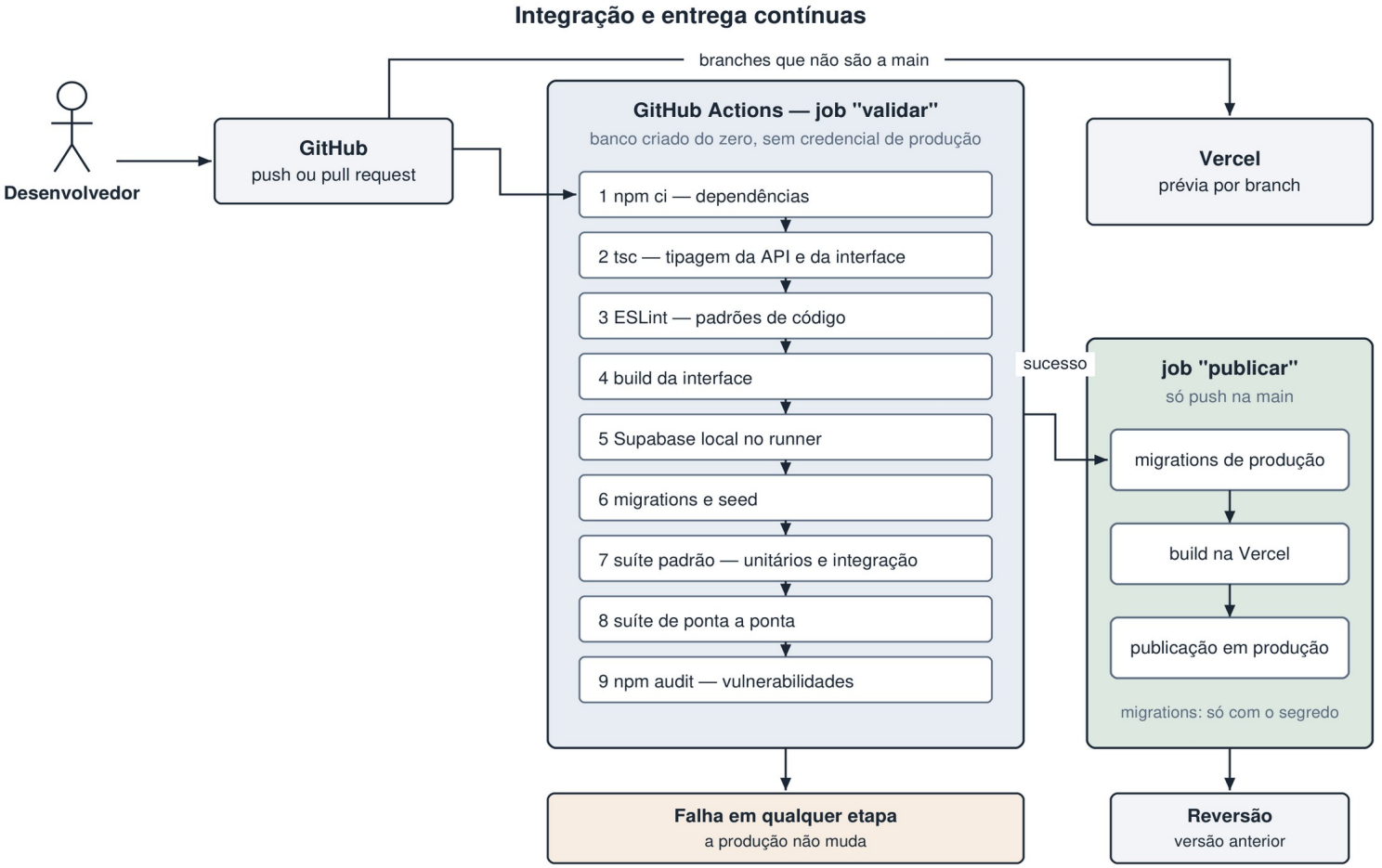
executadas pelas duas suítes juntas

0

vulnerabilidades

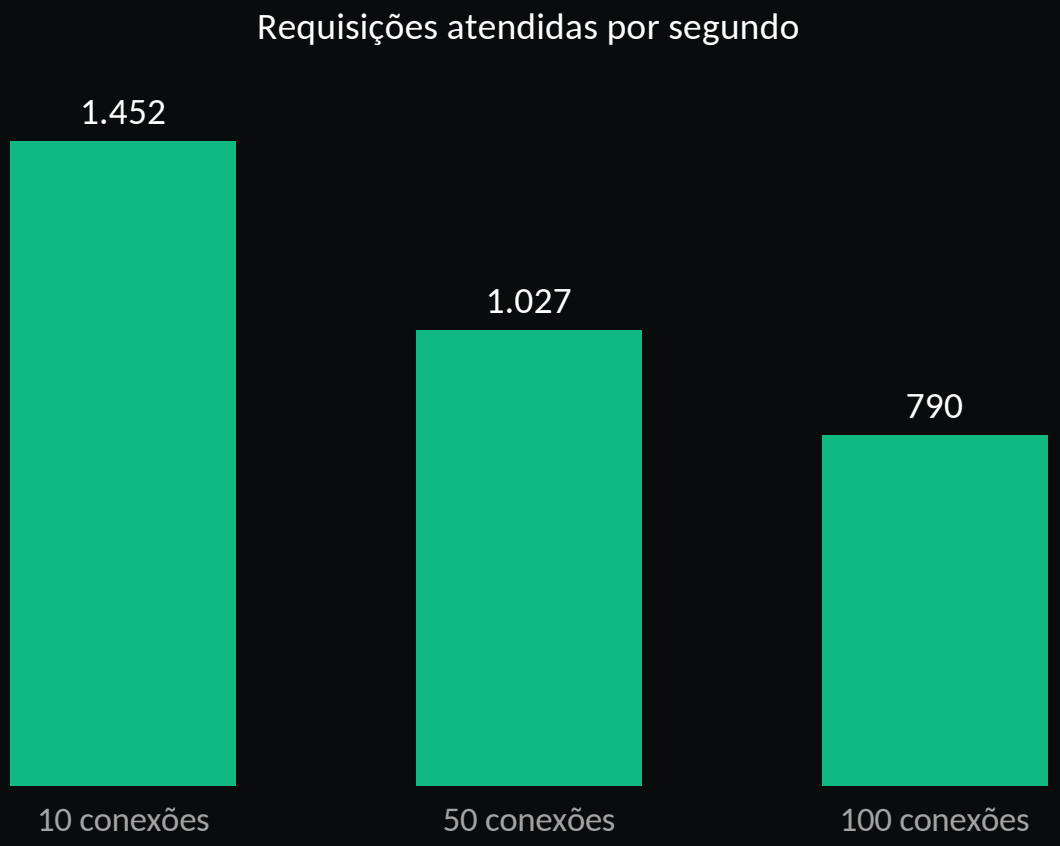
na auditoria de dependências

CI/CD: teste vermelho não vai ao ar



- Nove etapas a cada push e pull request, em cerca de 2 minutos
- Banco criado do zero; nenhuma credencial de produção
- A produção só é publicada depois que tudo passa
- Migrations aplicadas antes do código que depende delas

Desempenho: todas as rotas ao mesmo tempo



17 ms — p99 com 10 conexões

285 ms — p99 com 100 conexões

0 erros — em 124.753 requisições

Proporção: 60% lista de despesas · 15% lançamentos · 8% documentos · 8% saúde · 5% painel · 4% veículos

Volume e resiliência

50 mil despesas numa empresa

17 ms primeira página da lista

21 ms painel de 180 dias

0,69 s tela aberta no navegador

Banco desligado com a API no ar

1. **O banco cai**
   com a aplicação em uso
2. **Responde 503 em 3 s**
   em vez de travar
3. **Volta sozinho**
   primeira requisição atendida em 7 ms

Conclusão: cinco decisões que sustentam o sistema

1. Camadas com dependência em um só sentido, que tornam as regras testáveis
2. Autenticação delegada, autorização sob controle da aplicação
3. Tipos compartilhados entre interface e servidor
4. Empresa como fronteira de isolamento, em todo dado e toda consulta
5. Situações de prazo calculadas a cada consulta, nunca defasadas